Kasparro Candidate Screening & Ranking System

Production-Grade Automated Screening, GitHub Deep Profiling & Explainable 100-Point Scoring for SDE AI Roles

Official GitHub Repository: https://github.com/AveeckPandey/Kasparro
Project Author: Aveeck Pandey | **Tech Stack:** Python 3.10+, FastAPI, PyMuPDF, Pydantic v2, ReportLab, Rich CLI
Documentation Generated: October 2026 | **License:** Open Source / Educational

1. Executive Summary: What Does This Project Do?

The **Kasparro Candidate Screening & Ranking System** is an automated, explainable, production-ready recruitment pipeline specifically engineered to evaluate candidates for an **SDE Internship requiring strong Python backend fundamentals and practical exposure to Agentic AI and Retrieval-Augmented Generation (RAG)**.

Traditional Applicant Tracking Systems (ATS) rely on shallow keyword searches that can easily be manipulated by candidates copy-pasting buzzwords into their resumes. Kasparro replaces naive keyword scanning with an evidence-based multi-stage evaluation engine that deeply inspects document semantics, verifies code via public GitHub APIs, applies a 100-point rubric with penalties for shallow API wrappers, and synthesizes concise 50-100 word hiring justifications.

Key Core Capabilities:

- **Multi-Format Ingestion & Hyperlink Extraction:** Ingests PDF, DOCX, and TXT resumes. Uniquely extracts embedded PDF URI hyperlinks (such as clickable GitHub or LinkedIn icons) that do not expose raw URL text.
- **Hard Rule-Based Eligibility Gatekeeper:** Enforces non-negotiable baselines: requires verifiable Python backend experience (FastAPI, Flask, Django) and demonstrable AI/Agentic tooling (LangChain, LangGraph, ADK, RAG, vector search). Rejects pure JavaScript/Java candidates with explicit disqualification notes.
- **Deep GitHub Profiling & Developer Cohorts:** Queries GitHub's public API to calculate exact account age, count public and original repositories, measure recent commit velocity, and categorize developers into distinct cohorts (Veteran Builder, Active Builder, Emerging Developer, Dormant).
- **100-Point Explainable Scoring Rubric:** Evaluates candidates across 5 weighted dimensions (AI Depth: 40 pts, Python Backend: 30 pts, Cloud/Full Stack: 15 pts, GitHub: 10 pts, Engineering Signals: 5 pts) with automated penalties (-5 to -15 pts) for shallow LLM prompt-in/prompt-out wrappers.
- **Guaranteed 50-100 Word Narrative Justifications:** Employs an LLM Reasoning Engine (or deterministic heuristic fallback) to generate individualized, evidence-backed hiring justifications citing specific repositories and strengths.
- **Multi-Format Export Engine:** Automatically produces comprehensive exports in JSON, CSV, Microsoft Excel (.xlsx), and Microsoft Word (.docx).
- **Dual User Interfaces:** Operates both as a terminal CLI with rich styled tables and as an interactive, single-page web dashboard served by FastAPI.

2. Project Architecture & Directory Structure

The project follows a clean, modular, service-oriented architecture designed for maintainability, fault-tolerance, and high throughput. The repository tree and component responsibilities are outlined below:

```
Kasparro/
|-- main.py                    # Root CLI & FastAPI server entrypoint
|-- requirements.txt           # Production Python package dependencies
|-- .env.example               # Environment variables template & API keys
|-- .env                       # Local runtime configuration
|-- README.md                  # Project documentation & usage instructions
|-- static/
|   \-- index.html             # Single-page web dashboard (Monochrome dark UI)
|-- src/
|   |-- api.py                 # FastAPI server, screening routes & upload handlers
|   |-- cli.py                 # Rich terminal CLI engine with progress bars & tables
|   |-- config.py              # Pydantic Settings management & environment bindings
|   |-- models.py              # Strict Pydantic v2 data models, schemas & validation
|   |-- pipeline.py            # End-to-end multithreaded screening pipeline orchestrator
|   |-- parsers/
|   |   |-- resume_parser.py   # Multi-format doc parser (PyMuPDF, pypdf, docx) & link extractor
|   |   \-- entity_extractor.py # Regex & heuristic entity extractor (CGPA, skills, contact)
|   |-- filters/
|   |   \-- eligibility.py     # Hard rule-based eligibility gatekeeper
|   |-- enrichment/
|   |   \-- github_client.py   # GitHub REST client, rate-limit resilience, cache & cohorts
|   |-- scoring/
|   |   \-- scorer.py          # 100-point explainability scorer & shallow wrapper penalty
|   \-- llm/
|       \-- adapter.py         # Multi-provider LLM adapter (Groq, OpenAI, Gemini, Claude, Fallback)
|-- resumes/                   # Corpus of 50 sample candidate resumes (PDF, DOCX, TXT)
|-- output/                    # Generated results (results.json, .csv, .xlsx, .docx, cache)
\-- tests/                     # Automated test suite (Pytest fixtures & unit tests)
```

Module / File	Core Responsibility & Technical Implementation
`src/parsers/resume_parser.py`	Extracts text and embedded hyperlink annotations across PDF (PyMuPDF + pypdf fallback), DOCX (python-docx), and TXT formats. Never crashes on corrupted files.
`src/parsers/entity_extractor.py`	Normalizes candidate names, emails, phones, LinkedIn links, and CGPA (standardizes 10.0, 4.0 GPA, percentages to a uniform 10.0 scale). Categorizes skill tags across 6 technical domains.
`src/filters/eligibility.py`	Binary gatekeeper enforcing mandatory Python backend depth (FastAPI/Django) and demonstrable AI/Agentic project exposure. Produces explicit disqualified reasons.
`src/enrichment/github_client.py`	Queries public GitHub API to extract account age in years, public repositories, and commit velocity. Classifies candidates into developer cohorts with local caching to prevent rate limiting.
`src/scoring/scorer.py`	Implements the 100-point rubric. Rewards multi-agent graphs and hybrid RAG; deducts -5 to -15 pts for shallow wrappers and -8 pts for tutorial-style code.
`src/llm/adapter.py`	Provides unified adapter interface for Groq, OpenAI, Google Gemini, Anthropic Claude, and deterministic heuristic fallback. Enforces strict 50-100 word justification bounds.
`src/pipeline.py`	Orchestrates parallel candidate processing with ThreadPoolExecutor, aggregates batch metrics, and handles exports to JSON, CSV, Excel, and Word DOCX.
`src/api.py & static/index.html`	FastAPI service offering REST endpoints and serving the responsive single-page web UI with folder upload, PDF preview modal, live dials, and export toolbars.
`src/cli.py & main.py`	Command-line interface with rich ASCII banners, colored progress bars, summary metric tables, and server launch flags.

3. LLM Architecture: Which LLMs & Models Are Being Used?

Kasparro features a **pluggable, multi-provider LLM reasoning architecture** implemented via the BaseLLMAdapter pattern. It allows recruiters to switch between state-of-the-art hosted LLM providers or operate completely offline with zero external dependencies.

Provider	Default Model ID	API Protocol	Characteristics & Usage
Groq Cloud	qwen/qwen3.8-27b (or Llama 3.3 70B)	OpenAI-compatible HTTP (/v1/chat/completions)	Ultra-low latency inference (~250 tokens/sec). Ideal for processing high-volume batches (e.g. 50-100 resumes) in seconds at near-zero cost.
OpenAI	gpt-4o-mini (or gpt-4o)	OpenAI Chat Completions REST API	High reasoning precision and reliable instruction following. Used for nuanced evaluation justifications and technical stack verification.
Google Gemini	gemini-1.5-flash	Google Generative Language API (v1beta)	Fast multi-modal speed with long context capability and strong structural reasoning.
Anthropic Claude	claude-3-5-haiku-latest	Anthropic Messages REST API (/v1/messages)	High analytical fidelity and nuanced narrative generation. Excels at identifying shallow buzzwords vs genuine depth.
Heuristic Engine (Built-in Fallback)	Deterministic rule engine (MD5 Seed Rotation)	Local in-process Python execution (Zero network calls)	100% offline fallback. Employs MD5 candidate hashing to vary sentence structures and synthesize candidate-specific evidence. Zero latency, guaranteed 50-100 words.

Dynamic Provider Selection & Resilience Architecture:

- **Automatic Provider Discovery (LLM_PROVIDER=auto):** When set to auto, the factory inspects available environment variables in priority order (OPENAI_API_KEY -> GEMINI_API_KEY -> ANTHROPIC_API_KEY -> GROQ_API_KEY -> HeuristicAdapter).
- **Strict Anti-Template Negative Constraints:** Prompts explicitly prohibit generic openings (such as *'Candidate qualifies as an eligible candidate...'*) and enforce direct technical evidence citing specific repositories, score numbers, and technical gaps.
- **Guaranteed Word Count Enforcer:** The engine runs a programmatic _ensure_word_count() post-processor ensuring all narratives remain strictly within the 50 to 100 word constraint required by recruitment standards.
- **Graceful API Degradation:** If any LLM API call times out, encounters HTTP 429 rate limits, or returns non-conforming responses, it transparently falls back to the deterministic heuristic engine without aborting the batch run.

4. Explainable 100-Point Scoring Model & Penalties

Kasparro implements an objective, mathematically transparent 100-point rubric tailored for an SDE AI intern role. The score is decomposed into 5 distinct categories, complemented by deduction logic for low-effort submissions:

Category	Weight	Evaluation Criteria & Evidence Signals
AI / Agentic / RAG Depth	**40 pts**	LangGraph/LangChain multi-agent graphs, stateful agent loops, hybrid search RAG, vector databases (Chroma, Qdrant, Pinecone), embeddings, tool calling, and evaluation pipelines.
Python & Backend Engineering	**30 pts**	Modern async FastAPI/Django services, SQLAlchemy ORM, PostgreSQL database schemas, Redis caching, and async worker patterns. Mentions without depth receive capped credit.
Cloud, DevOps & Full Stack	**15 pts**	Containerization (Docker, Docker Compose), cloud deployment (AWS, GCP), CI/CD pipelines, and supporting modern web frontends (Next.js, React).
GitHub Activity & Proven Code	**10 pts**	Scored from verified public activity: account longevity (0-2 pts), public repository depth (0-3 pts), and recent commit velocity/pulse (0-5 pts).
Engineering Depth Signals	**5 pts**	Unit and integration testing (pytest, unittest), message queues (Celery, Kafka, RabbitMQ), logging, telemetry, and observability.

Automated Penalty Architecture:

- **Shallow LLM Wrapper Penalty (-5 to -15 pts):** Penalizes resumes that claim 'AI development' but only implement single-prompt OpenAI API completions without state management, tool calling, vector search, or evaluation.
- **Tutorial / Forked Clone Penalty (-8 pts):** Penalizes obvious tutorial clones (e.g. basic to-do apps with a chatbot) that demonstrate zero independent architecture or engineering ownership.

5. Step-by-Step Guide: How to Run This Project

Step 1: Clone the Repository

```
git clone https://github.com/AveeckPandey/Kasparro.git
cd Kasparro
```

Step 2: Create a Virtual Environment & Install Dependencies

```
# Create virtual environment
python -m venv venv

# Activate environment
# Windows (PowerShell):
.\venv\Scripts\activate
# macOS / Linux:
source venv/bin/activate

# Install required dependencies
pip install -r requirements.txt
```

Step 3: Configure Environment Variables (Optional)

Copy the provided template to configure API keys for GitHub or LLM providers:

```
cp .env.example .env

# Optional variables in .env:
# GITHUB_TOKEN=ghp_your_token_here       # Increases GitHub API rate limit from 60 to 5,000 req/hr
# LLM_PROVIDER=auto                      # auto, groq, openai, gemini, anthropic, or heuristic
# GROQ_API_KEY=gsk_your_groq_key_here    # Free ultra-fast Qwen/Llama inference
# OPENAI_API_KEY=sk-your_openai_key_here # Optional OpenAI API key
```

Step 4: Launch the Web Dashboard (Interactive Interface)

Launch the FastAPI web server to access the sleek monochrome recruiter dashboard:

```
python main.py --serve --port 8000

# Open your web browser at:
# http://localhost:8000
```

Web UI Capabilities: Drag-and-drop batch resume upload (up to 100 files), 1-click sample screening of the 50 bundled resumes, interactive min passing score dialer, Top 5/10/20 quick filter pills, real-time candidate search, PDF resume viewer modal, and one-click downloads for Excel, Word, CSV, and JSON.

Step 5: Run Batch Screening via Command-Line Interface (CLI)

For CI/CD pipelines or headless server workflows, execute screening directly via the CLI:

```
# Standard batch screening on sample resumes
python main.py --input ./resumes --output ./output/results.json

# Filter with a minimum CGPA threshold (e.g. 7.5 on 10.0 scale)
python main.py --input ./resumes --min-cgpa 7.5

# Require verified industry experience and a public GitHub profile link
python main.py --input ./resumes --require-experience --require-github

# Specify multi-threaded concurrency (default: 5 worker threads)
python main.py --input ./resumes --concurrency 8 --format json,csv
```

CLI Arguments Reference:

Flag	Type	Default	Description
`--input, -i`	string	`./resumes`	Path to directory containing resumes (PDF, DOCX, TXT).
`--output, -o`	string	`./output/results.json`	Destination path for structured output JSON.
`--format, -f`	string	`json,csv`	Comma-separated export formats (json, csv).
`--min-cgpa`	float	`None`	Filter candidates below this CGPA (on 10.0 scale).
`--require-experience`	flag	`False`	Filter strictly for candidates with related work experience.
`--require-github`	flag	`False`	Require presence of a public GitHub profile link.
`--concurrency`	int	`5`	Number of parallel worker threads.
`--serve`	flag	`False`	Launch FastAPI web server instead of headless CLI.
`--port`	int	`8000`	HTTP port for web dashboard.

Step 6: Running Automated Tests

```
# Run full test suite
pytest -v

# Run tests with output logging
pytest -v -s
```

6. REST API Endpoints Reference

Endpoint	Method	Payload / Params	Response & Function
`/`	GET	None	Serves the single-page web dashboard HTML.
`/health`	GET	None	Health check returning service status (healthy).
`/screen`	POST	`JSON: {input_dir, min_cgpa, require_ai...}`	Executes screening on a specified server directory and returns the full screening report.
`/upload_and_screen`	POST	Multipart Form: files (up to 100) + filters	Uploads files, runs screening immediately, and saves originals for recruiter preview.
`/results`	GET	None	Retrieves the cached screening report from the most recent run.
`/resumes/{file_name}`	GET	Path param: resume filename	Direct file access for the built-in PDF iframe modal viewer.
`/export_filtered_excel`	POST	`JSON: {candidates, filter_label}`	Dynamically generates and downloads an Excel spreadsheet for the active shortlist view.
`/export_filtered_docx`	POST	`JSON: {candidates, filter_label}`	Dynamically generates and downloads an executive Word DOCX report for the current view.

7. Structured Deliverables & Export Formats

Kasparro ensures end-to-end recruiter interoperability by exporting structured screening reports across 4 distinct formats:

- **JSON Report (output/results.json):** Comprehensive schema containing batch summary statistics, ranked candidates with nested score breakdowns and GitHub metrics, and rejected candidates with explicit disqualification reasons.
- **CSV Export (output/results.csv):** Flat tabular export containing all candidate fields, contact details, individual rubric sub-scores, and 50-100 word justifications for easy integration into spreadsheets or ATS databases.
- **Microsoft Excel (output/results.xlsx):** Styled multi-sheet workbook with custom color headers, auto-fit column widths, frozen header panes, and formatted numbers.
- **Microsoft Word (output/results.docx):** Clean, professionally formatted executive summary document suitable for forwarding directly to hiring engineering managers.

8. Summary & Project Links

The **Kasparro Candidate Screening & Ranking System** bridges the gap between high-volume hiring and rigorous technical evaluation. By combining strict rule-based eligibility checks, automated GitHub code verification, a 100-point explainable scoring rubric, and high-speed LLM reasoning, it empowers recruitment teams to shortlist top-tier SDE AI talent with speed, consistency, and complete transparency.

Source Code & Contributions:
Explore the source code, open issues, and star the project on GitHub:
https://github.com/AveeckPandey/Kasparro